

project1 - report

2022036208 인범진

Design

- xv6의 Round Robin Scheduler를 **Completely Fair Scheduler**로 재구성하였다
 - RR이 time quantum에 입각하여 scheduling을 하는 반면, CFS는 정해진 우선순위에 따라 시간을 할당한다
- 제시된 **Red-Black Tree** 자료구조를 process들 중 CPU를 할당할 process selection에 이용했다
 - 명세를 보면, **"Select the RUNNABLE process with the lowest virtual time"**이라는 표현이 있다
 - 즉, 항상 vruntime의 **최솟값**을 가지는 process를 찾아야 한다
 - red-black tree에서 `rb_first()`의 구조를 보면 가장 왼쪽의 노드를 반환하는데, 이는 **최솟값**을 가지는 노드를 의미한다

```
struct rb_node* rb_first(struct rb_node *node) { // 최소값
    if (node == 0) return 0;
    while (node->left != 0)
        node = node->left;
    return node;
}
```

- 또한 `rb_node` 구조체에서 `key` 값을 트리 정렬의 기준이 되는 vruntime으로, `data`를 proc 포인터 값으로 이용할 수 있었다
- 구현 순서를 정리해보면 다음과 같다
 - red-black-tree를 프로세스의 자료구조에 적용
 - kernel/proc.h의 struct proc에 새로운 필드(vruntime, nice, rbnode) 추가
 - kernel/proc.c에 전역변수 `struct rb_tree runnable_tree` 선언
 - kernel/proc.c의 scheduling 관련 함수 수정
 - `procinit()`, `userinit()` → 초기화 관련 함수 수정
 - `scheduler()` 수정
 - `kfork()` 수정
 - `wakeup()` 수정
 - `kkill()` 수정
 - `setnice()` system call 구현
 - multi CPU 관련 추가 검토
 - 기존의 구현에서 Single CPU에만 잘 적용되고, Multi CPU에서 기대한 결과가 나오지 않았음
 - 이에 대한 코드를 보완하여, 문제를 해결함
 - 이에 대한 내용은 troubleshooting에서 밝히고자 함
 - usertest로 결과 검증
 - 정리하면 kernel에서 proc.h와 proc.c를 수정하여 process의 자료구조에 red_black_tree를 활용할 수 있도록 하였고, 이후 proc.c에서 scheduling과 관련된 함수 코드를 수정하였고, 마지막으로 setnice를 구현한 뒤 usertest로 검증하여 이 project를 마무리 했다

Implementation

- red-black-tree를 프로세스 자료구조에 적용

- CFS를 red_black tree로 구현하기 위해 proc.h에 다음의 코드를 추가했다

```
// Per-process state
struct proc {
    struct spinlock lock;

    ... (중략) ...

    // 새로 추가한 값
    uint64 vruntime;           // virtual runtime
    int nice;                  // Nice value
    struct rb_node rbnode;     // Red-Black tree node
};
```

- 각 process의 우선순위와 관련된 vruntime, nice value, 그리고 red black tree node를 새롭게 추가했다
- red black tree node를 사용하기 위해 `#include "rbtree.h"`를 코드 상단에 추가했다
- 이후에 proc.c에서 `rb_tree` 전역변수를 선언하여, process들이 담길 tree를 만들었다

```
struct rb_tree runnable_tree;
```

- 또한 멀티 cpu가 동시에 접근하는 상황을 방지하기 위해 spinlock을 이용했다

```
struct spinlock tree_lock;
```

- 앞서 lab06에서 활용한 queue에서는 queue 내부에 spinlock이 선언돼 있었으나, red_black_tree의 경우에 그렇지 않으므로, proc.c에 선언해주었다
- 처음 구현할 때에 이 spinlock을 적용하지 않아서, Multi processor 상황에서 문제가 발생했다

- kernel/proc.c의 scheduling 관련 함수 수정

`procinit()`, `userinit()` → 초기화 관련 함수 수정

- lab06에서 다른 queue 자료구조와 달리 `rb_tree`는 별도의 초기화 함수가 없었다
- 따라서 명시적으로 초기화를 해주었다

```
// initialize the proc table.
void
procinit(void)
{
    struct proc *p;

    initlock(&pid_lock, "nextpid");
    initlock(&wait_lock, "wait_lock");

    initlock(&tree_lock, "rbtree");
    runnable_tree.root = 0; // root 값
    이 0이면 빈트리로 인식
```

```
// Set up first user process.
void
userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    p->cwd = namei("/");

    p->state = RUNNABLE;
```

```

for(p = proc; p < &proc[NPROC]; p++) {
    initlock(&p->lock, "proc");

    p->state = UNUSED;
    p->kstack = KSTACK((int) (p - proc));
}
}

```

```

p->nice = 0;
p->vruntime = 0;

p->rbnode.key = 0;
p->rbnode.data = p;
rb_insert(&runnable_tree, &p->rbnode);
release(&p->lock);
}

```

- `procinit()` 를 수정하는 과정에서, `rb_first()` 와 `rb_insert()` 코드를 보고 root가 0일때 빈 트리 인식함을 알 수 있었다
 - 따라서 root 값을 0으로 초기화했다
 - `initlock()` 함수로 `tree_lock` 을 초기화했다
- `userinit()` 를 수정할 때에는 명세의 초기화 관련 언급을 바탕으로 process의 변수들을 초기화했다
 - 첫 process이므로 부모가 없으므로 vruntime을 0으로, nice값을 0으로 초기화했다
 - `rbnode->key` 값이 앞서 언급한 것처럼 vruntime, `rbnode->data` 값이 process 포인터 값이 된다
 - 그리고 runnable_tree에 `rb_insert()` 로 이 노드를 삽입했다

`scheduler()` 수정

- 기존의 RR 방식은 for문으로 process 배열을 순회하고, 순회할 때마다 state가 `RUNNABLE` 인 process를 찾아 실행시키는 방식이었다
- 우리의 CFS 방식은 red black tree에서 (우선순위 정책에 따라) 가장 낮은 vruntime을 가지는 process(노드)를 가져와서, 이 process를 실행시킨다

```

void
sscheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        intr_on();
        intr_off();

        int found = 0;

        acquire(&tree_lock);
        struct rb_node *node = rb_first(runnable_tree.root);

        if(node != 0){
            p = (struct proc *)node->data;

            acquire(&p->lock);

            if(p->state == RUNNABLE) {

```

```

// 실행 전에 트리에서 제거해야 함
// 꺼내기 전에 실행하면 다른 CPU가 같은 프로세스 선택할 수 있음!
rb_delete(&runnable_tree, &p->rbnode);
release(&tree_lock);

p->state = RUNNING;
c->proc = p;
swtch(&c->context, &p->context);

c->proc = 0;
found = 1;

// 실행 후 vruntime update

p->vruntime += p->nice >= 0 ? (1024 << p->nice) : (1024 >> -(p->nice));

if(p->state == RUNNABLE) {
    acquire(&tree_lock);
    p->rbnode.key = p->vruntime; // ← 업데이트된 값으로!
    p->rbnode.data = (void *)p;
    rb_insert(&runnable_tree, &p->rbnode);
    release(&tree_lock);
}
}else {
    release(&tree_lock);
}

release(&p->lock);

} else{
    release(&tree_lock);
}

if(found == 0) {
    asm volatile("wfi");
}
}
}

```

- red black tree에서 `rb_first()` 로 노드를 꺼내면, 자연스럽게 가장 낮은 vruntime을 갖는 노드가 나온다
- 이 프로세스의 상태를 확인한 후 `RUNNABLE` 이 맞다면, 스케줄링의 대상이 된다
- 프로세스 실행 이후 vruntime을 update 하는 코드를 추가했다

```

p->vruntime += p->nice >= 0 ? (1024 << p->nice) : (1024 >> -(p->nice));

```

- vruntime의 값은 1씩 증가할 때마다 2배씩 커진다
- 이때, 나는 vruntime을 정해진 단위가 있는 값으로 오해했는데, nice value에 따라 정해주기 나름이었다
- 따라서 1024 라는 값에 "`<<`" (left shift operator)"와 "`>>`" (right shift operator)"을 `p->nice` 값과 연산하여 나온 값을 vruntime에 더해주었다.
- 해당 프로세스 실행(swtch함수 실행) 후에 vruntime을 update했으므로, 다시 트리에 삽입해줘야 한다

- 이때 프로세스의 상태는 `RUNNABLE`, `SLEEPING`, `ZOMBIE` 중 하나가 된다
- 만약 lab06에서 queue를 이용해 while문을 사용했던 것 처럼, 여기서도 while문을 사용하면, 같은 프로세스가 계속 실행되므로 우리가 정한 우선순위 정책에 위반된다
- `tree_lock` 을 이용하여 `rb_first()` 와 `rb_delete()` 사이 CPU의 동시 접근을 차단한다

`kfork()` 수정

```
// Create a new process, copying the parent.
int
kfork(void)
{
    int i, pid;
    struct proc *np; // 새로 생성될 자식의 process 포인터
    struct proc *p = myproc(); // 현재 실행중인 부모의 process 포인터

    ... (중략) ...

    np->nice = p->nice;          // 부모 nice 상속
    np->vruntime = p->vruntime; // 부모 vruntime 상속
    np->rbnode.key = np->vruntime;
    np->rbnode.data = (void *)np;
    rb_insert(&runnable_tree, &np->rbnode);

    release(&np->lock);

    return pid;
}
```

- 명세에 따라 fork 된 자식 process는 부모의 vruntime과 nice를 상속받는다
- `np` 가 가지는 rbnode의 key값과 data를 각각 vruntime, np(자기 자신의 포인터)값으로 할당한다
- 그렇게 세팅한 자식 process(`np`)의 rbnode를 `rb_insert()` 함수로 tree에 추가한다

`wakeup()` 수정

```
// Wake up all processes sleeping on channel chan.
void
wakeup(void *chan)
{
    struct proc *p;
    struct proc *pp;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            acquire(&p->lock);
            if(p->state == SLEEPING && p->chan == chan) {

                acquire(&tree_lock);
                struct rb_node *minnode = rb_first(runnable_tree.root);
```

```

uint64 min_running = (uint64)-1; // uint 최대값
for(pp = proc; pp < &proc[NPROC]; pp++){
    if(pp->state == RUNNING && pp->vruntime < min_running)
        min_running = pp->vruntime;
}

if(minnode){ // 1 RUNNABLE 최소 vruntime 찾기
    p->vruntime = minnode->key;
} else if (min_running != (uint64)-1){ // 2 RUNNABLE 없으면 RUNNING 찾기
    p->vruntime = min_running;
} else { // 3 둘다 아니면 0으로
    p->vruntime = 0;
}

p->state = RUNNABLE;
p->rbnode.key = p->vruntime;
p->rbnode.data = (void *)p;
rb_insert(&runnable_tree, &p->rbnode);
release(&tree_lock);
}

release(&p->lock);
}
}
}

```

- 앞에서와 마찬가지로 `tree_lock()` 으로 `runnable_tree` 에 대한 동시 접근을 차단한다
- 과제 명세에 따라 `SLEEPING` process가 깨어나면(`wakeup()` 이 호출되면), `vruntime`을 update하는 데, 세가지 규칙을 따른다
 - `RUNNABLE` process가 존재하는 경우, `RUNNABLE` 없이 `RUNNING` 만 존재하는 경우, 둘다 아닌 경우로 구분한다
 - `minnode` 에 현재 트리에서 최소 `vruntime`을 가지는 노드를 `rb_first()` 로 가져온다(이때 노드가 없으면 0을 반환)
 - `min_running` 은 `RUNNING` process 중 최소 `vruntime`을 가지는 노드의 `vruntime`을 저장하는 용도이다
 - 만약 이 값이 update 되지 않는다면 `RUNNING` process가 없는 것이다
 - `struct proc *pp` 를 추가하여, `RUNNING` process 중 최소 `vruntime`을 찾기위해 순회할 때 사용한다
- 깨어난 process의 상태를 `RUNNABLE` 로 전환한다(스케줄링의 대상으로 만듦)
- 이러한 규칙에 따라 정해진 `vruntime`으로 process의 `rbnode`도 update 해준다

```

p->rbnode.key = p->vruntime;
p->rbnode.data = (void *)p;
rb_insert(&runnable_tree, &p->rbnode);

```

`kill()` 수정

```

// Kill the process with the given pid.
int

```

```

kkill(int pid)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            if(p->state == SLEEPING){
                p->state = RUNNABLE;

                // wakeup과 동일하게 vruntime 설정 후 rb_insert
                struct rb_node *minnode = rb_first(runnable_tree.root);
                if(minnode)
                    p->vruntime = minnode->key;
            }
            else {
                // RUNNING 중 최소 vruntime 찾기
                uint64 min_running = (uint64)-1;
                struct proc *pp;
                for(pp = proc; pp < &proc[NPROC]; pp++) {
                    if(pp->state == RUNNING && pp->vruntime < min_running)
                        min_running = pp->vruntime;
                }
                p->vruntime = (min_running != (uint64)-1) ? min_running : 0;
            }

            p->rbnode.key = p->vruntime;
            p->rbnode.data = (void *)p;
            rb_insert(&runnable_tree, &p->rbnode);
        }
        release(&p->lock);
        return 0;
    }
    release(&p->lock);
    return -1;
}

```

- `wakeup()` 과 마찬가지로 `tree_lock`으로 보호한다
- `wakeup()` 과 동일한 `vruntime` update방식을 적용한다

setnice() system call 구현

- `set_nice()` 시스템 콜은 이전 `ps` 시스템 콜 구현과 동일한 방식으로 구현했다

[kernel 파일]

- `set_nice`라는 함수를 `kernel/proc.c`에 구현함
- `kernel/defs.h`에 다른 `c`파일 처럼 `visible`하게 만들
- `kernel/sysproc.c`에 `ps` system call을 위한 `wrapper function`을 만들
- `kernel/syscall.h`에 `wrapper function number`를 등록함
- 이를 통해 `kernel/syscall.c`에서 `wrapper function`을 참조함

[User 파일]

- user/usys.pl 코드를 추가하여, `set_nice()` 호출을 통해 `set_nice()` 시스템 콜 번호를 레지스터에 넣고, `ecall`을 실행함
- `ps` 시스템 콜 구현과 다른 점은 다음과 같다
- **kernel/proc.c - `set_nice()` 구현**
 - `ps`는 `proc` 배열을 순회하며 출력하는 함수였던 반면, `set_nice()`는 현재 실행 중인 프로세스의 `nice` 값을 설정하는 함수이다
 - 명세에 따라 `nice` 값의 유효 범위(-3 ~ 2)를 검증한 후 설정하며, 범위를 벗어나면 1을 반환한다

```
int set_nice(int nice) // nice value를 세팅하는 함수
{
    struct proc *p = myproc();

    if(nice < -3 || nice > 2)
        return 1; // nice value가 invalid
    else{
        acquire(&p->lock);
        p->nice = nice;
        release(&p->lock);
        return 0; // nice value가 성공적으로 update
    }
}
```

- `ps`와 달리 `p->lock`을 획득한 후 `nice`를 설정한다
 - 이는 멀티 CPU 환경에서 다른 CPU가 동시에 같은 프로세스의 `nice` 값을 변경하는 상황(race condition)을 방지하기 위함이다

kernel/sysproc.c - `sys_set_nice()` 구현

```
uint64
sys_set_nice(void)
{
    int nice;
    argint(0, &nice);
    return set_nice(nice);
}
```

- `ps`의 `sys_ps()`와 동일한 구조로, `argint()`로 유저로부터 인자를 받아 kernel 함수로 전달한다.

Results

first usertest result

- `make qemu CPUS=1`로 Single Processor 환경을 점검한다
- user test는 Discussion에 올라온 학우님들의 usertest를 활용했다

```

inbeomjin@inbeomjin-ui-MacBookAir: ~/project1-Ben9960 / project1 ± make qemu CPUS=1
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -marc
h=rv64gc -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -f
no-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-b
zero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-buili
n-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -l. -fno-stack-protector -fno-
pie -no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/start.
o kernel/console.o kernel/printf.o kernel/uart.o kernel/kalloc.o kernel/spinlock.o kernel/string.o kernel/main
.o kernel/vm.o kernel/proc.o kernel/switch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.
o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sy
sfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/rbtree.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .*/ /; / ^$/d' > kernel/kernel.sym
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mm
io.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virti
o-mmio-bus.0

```

```

$ cfs_test
=====
CFS Scheduler Test Suite
(CPUS=1 mode)
=====

=== Test 1: set_nice boundary ===
[PASS] set_nice(-3) returns 0
[PASS] set_nice(-2) returns 0
[PASS] set_nice(-1) returns 0
[PASS] set_nice(0) returns 0
[PASS] set_nice(1) returns 0
[PASS] set_nice(2) returns 0
[PASS] set_nice(-4) returns 1
[PASS] set_nice(3) returns 1
[PASS] set_nice(-100) returns 1
[PASS] set_nice(100) returns 1

=== Test 2: nice inheritance via fork ===
[PASS] child inherits parent nice and validates correctly
[PASS] child inherits nice=2, rejects nice=3

=== Test 3: initial nice value ===
[PASS] process starts with valid nice=0

=== Test 4: scheduling order by nice value ===
Expected order: nice=-3 finishes first, nice=2 finishes last
Completion order: -3 0 2
[PASS] lower nice finishes before higher nice

=== Test 5: vruntime inheritance (CPUS=1) ===
Completion order: X then Y
[PASS] lower vruntime child (X) finishes before higher vruntime child (Y)

=== Test 6: wakeup vruntime update ===
woken process completed
[PASS] process wakes from sleep and continues execution

=== Test 7: multi-process stability (CPUS=1) ===
[PASS] 6 processes with different nice values all complete

=== Test 8: nice inheritance chain ===
[PASS] nice inheritance works through fork chain

```

```

=== Test 9: kill sleeping process ===
[PASS] kill sleeping process does not crash system

=== Test 10: single process basic operation ===
[PASS] single child process runs and exits cleanly

=====
PASS: 20 / FAIL: 0
ALL TESTS PASSED

```

◦ 다음의 실행결과를 보면 과제 명세가 요구하는 여러 조건에 맞는지를 확인할 수 있었다

1. set_nice() 리턴값 검증
2. fork 시 Nice & vruntime 상속하는지 검증
3. wakeup할 때 minimum vruntime으로 설정하는지 검증
4. nice value값을 기준으로 스케줄링을 하는지

5. kill된 SLEEPING process 처리
6. muti processor 환경에서 deadlock이 발생하는지 여부 검증

Second usertest result

- 두번째 usertest는 구현한 스케줄링이 기대한 대로 동작하는지를, 보다 구체적으로 확인할 수 있었다

[illegible]

Third usertest

- 세번째 usertest에서는 CPU가 2개인 경우로 세팅하여, 실행하였다
- cpu가 두개인 상황에서도 실행이 잘 되는 것을 확인할 수 있었다

rb_delete 인자 오류

- 초기 구현 시 `rb_delete()` 의 두 번째 인자로 `rb_first(runnable_tree.root)` 를 넘겼다

```
// 초기 구현
rb_delete(&runnable_tree, rb_first(runnable_tree.root));
```

- `swtch()` 이후 `root`가 바뀌었을 수 있으므로 `rb_first()` 가 반환하는 노드가 실제로 실행한 프로세스의 노드가 아닐 수 있다
- 따라서, 실행한 프로세스의 `rbnode`를 직접 넘기도록 수정하였다

```
// 수정된 구현
rb_delete(&runnable_tree, &p->rbnode);
```

vruntime 단위 오해

- 처음 구현 시에 `vruntime`을 tick 단위의 절대적인 실행 시간으로 잘못 이해하였다
- 이로 인해 `vruntime` 증가량 계산 시 실제 실행 시간을 측정하려 했으나 `xv6 kernel`에서 정확한 실행 시간 측정이 어려워 구현에 혼란이 생겼다
- `vruntime`은 절대적인 단위가 아니라 프로세스 간 상대적 비율만 유지하면 된다는 것을 파악하였다
- 따라서 시간의 기본 단위를 1024로 설정하여 다음과 같이 계산하였다

```
p->vruntime += p->nice >= 0 ? (1024 << p->nice) : (1024 >> -(p->nice));
```

- 이 방식을 이용하여 음수 `nice`도 정수 연산으로 표현 가능하며, `nice` 값이 1 증가할 때마다 증가량이 2배가 되는 명세를 충족할 수 있었다

Multi CPU 관련 추가 검토

- 초기 구현 시 `runnable_tree`에 대한 별도의 `lock`을 추가하지 않았다
- `CPUS=1` 환경에서는 문제가 없었으나, 멀티 CPU 환경에서는 여러 CPU가 동시에 `runnable_tree`에 접근할 경우 `race condition`이 발생할 수 있었다

```
CPU 0: rb_first() → 같은 노드 선택
CPU 1: rb_first() → 같은 노드 선택
CPU 0: rb_delete() → 노드 삭제
CPU 1: rb_delete() → 이미 삭제된 노드 삭제 !
```

- 또한 초기 `tree_lock` 추가 시 `acquire/release` 균형이 맞지 않아 `panic: release` 오류가 발생하였다. `node`가 없을 때 `acquire`한 `tree_lock`을 `release`하지 않는 경우가 있었다.
- 따라서, 전역 `spinlock tree_lock`을 추가하고 모든 `runnable_tree` 접근 시 보호하도록 하였다.
- `acquire/release` 균형을 위해 모든 분기에서 반드시 `release`가 호출되도록 구조를 정리하였다.